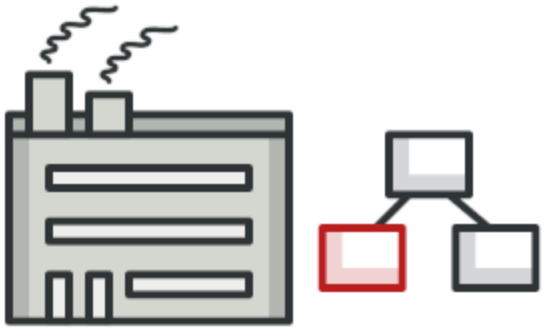
Порождающие паттерны. Фабричный метод

Паттерн Фабричный метод
Factory Method

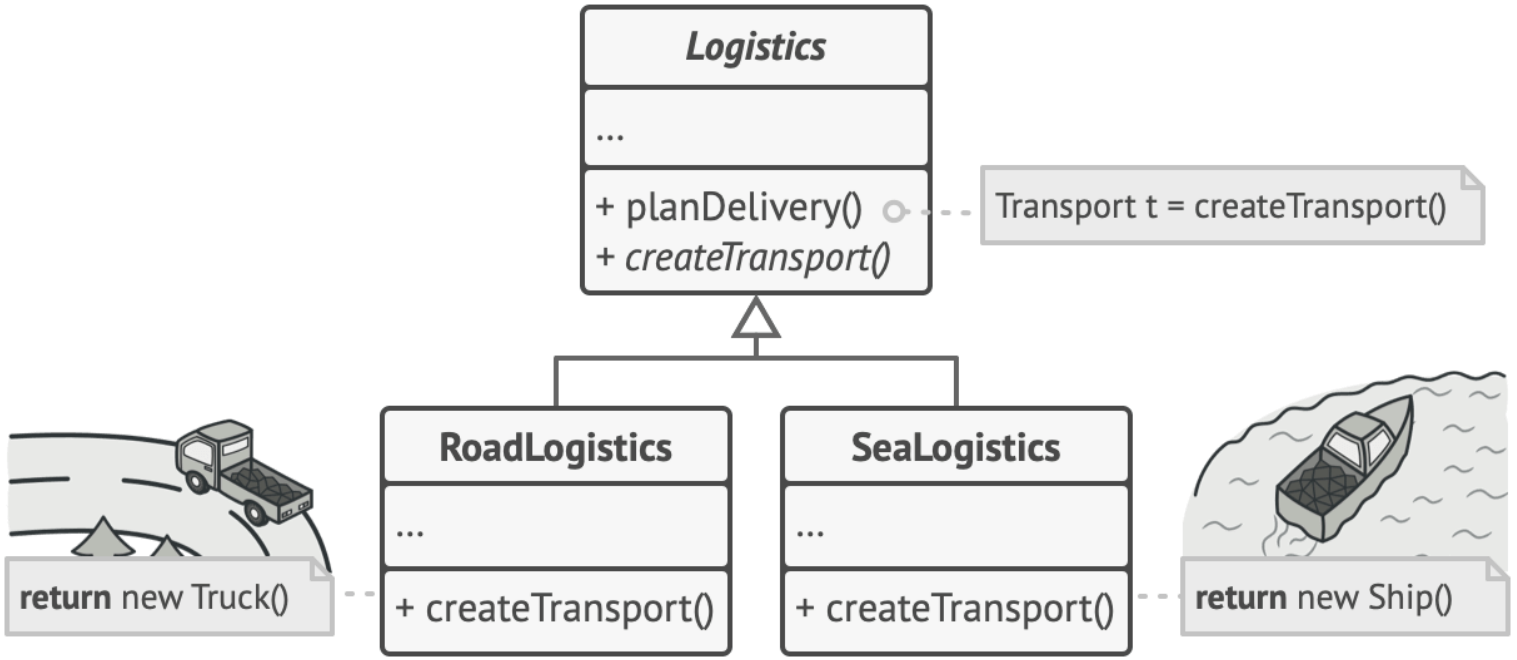


Фабричный метод — это порождающий паттерн проектирования, который в суперклассе определяет общий интерфейс для создания объектов, позволяя подклассам изменять тип создаваемых объектов.

- Паттерн **Factory Method** лежит в основе всех порождающих паттернов, организовывая процесс порождения объектов-продуктов. Если проектировщик на этапе проектирования системы не может сразу определиться с выбором подходящего паттерна для организации процесса порождения продукта в конкретной ситуации, то сперва следует воспользоваться паттерном **Factory Method**.
- **Factory Method** является паттерном уровня классов, реализует отношения между классами.
- Основной задачей паттерна **Factory Method** является организация техники *делегирования ответственности* за создание объектов продуктов одним классом (*абстрактным*) другому классу (*производному конкретному классу*).

Фабричный метод

Паттерн **Фабричный метод** предлагает создавать объекты не напрямую, используя оператор **new**, а через вызов *особого фабричного метода*.



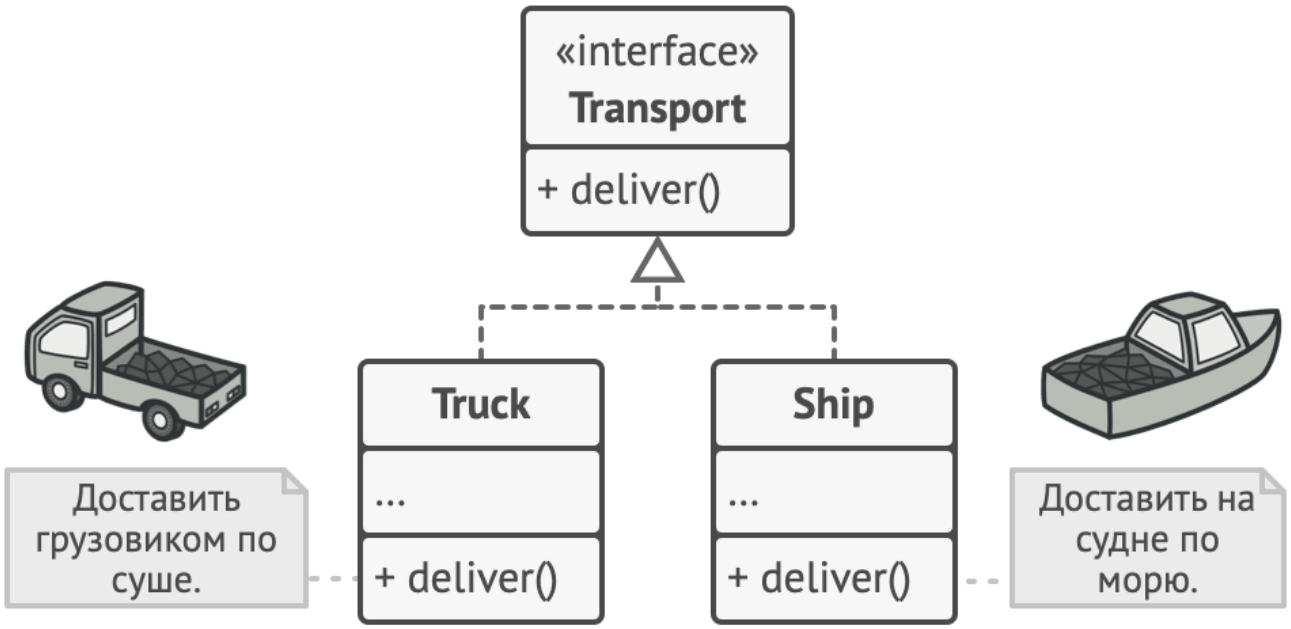
Подклассы могут изменять класс создаваемых объектов.

(объекты создаются при помощи new, но занимается этим ***фабричный метод***)

Появляется возможность переопределить фабричный метод в подклассе, чтобы изменить тип создаваемого продукта.

Фабричный метод

- Все возвращаемые объекты должны иметь общий интерфейс.
- Подклассы смогут производить объекты различных классов, следующих одному и тому же интерфейсу.



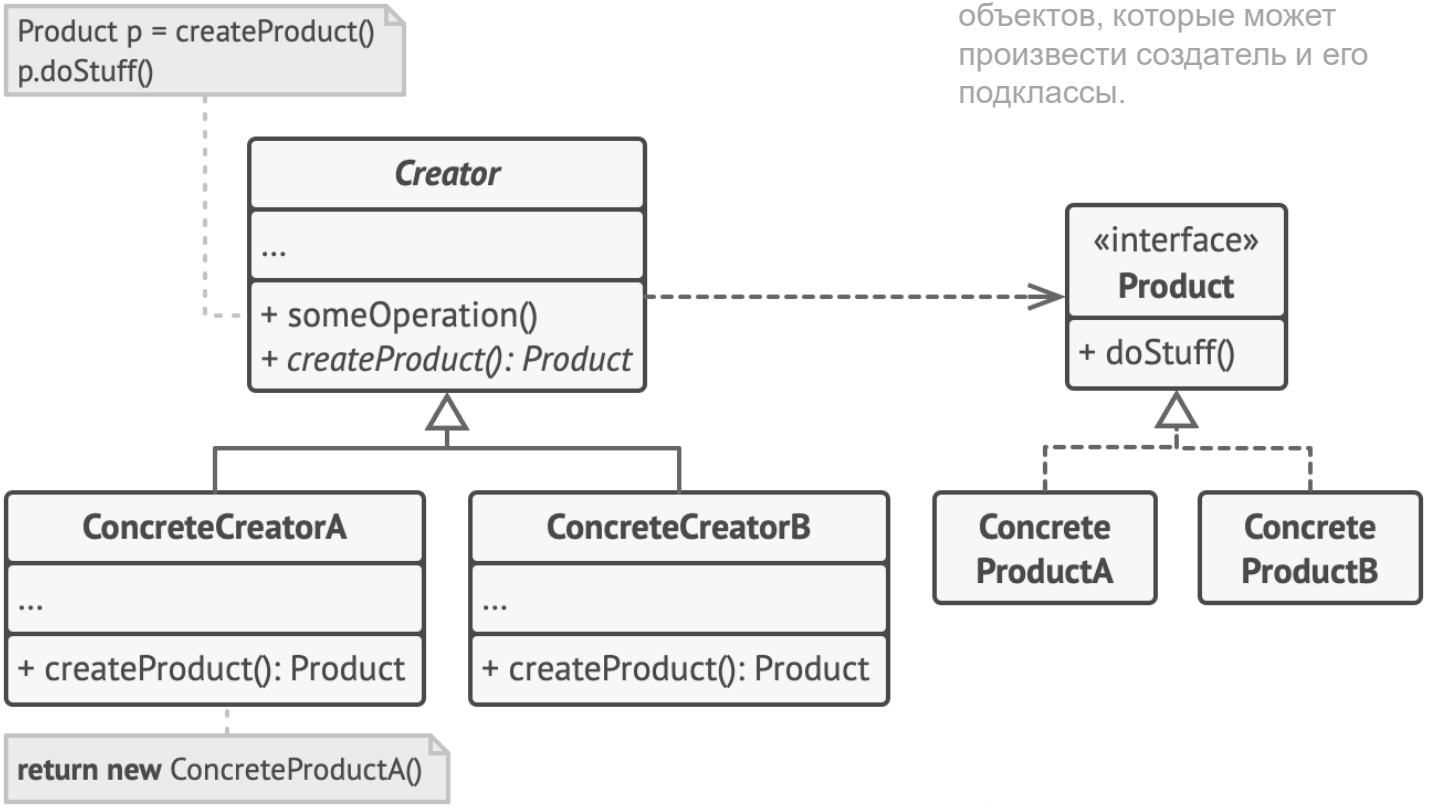
Для клиента фабричного метода нет разницы между конкретными объектами, так как он будет трактовать их как некий *абстрактный Транспорт*.
Важно, чтобы объект имел метод *доставить*, а особенности его реализации неважны.

Структура паттерна Фабричный метод

[3]**Создатель** объявляет фабричный метод, который должен возвращать новые объекты продуктов. Важно, чтобы тип результата совпадал с общим интерфейсом продуктов.

Зачастую фабричный метод объявляют абстрактным, чтобы заставить все подклассы реализовать его по-своему. Но он может возвращать и некий стандартный продукт.

[1]**Продукт** определяет общий интерфейс объектов, которые может произвести создатель и его подклассы.



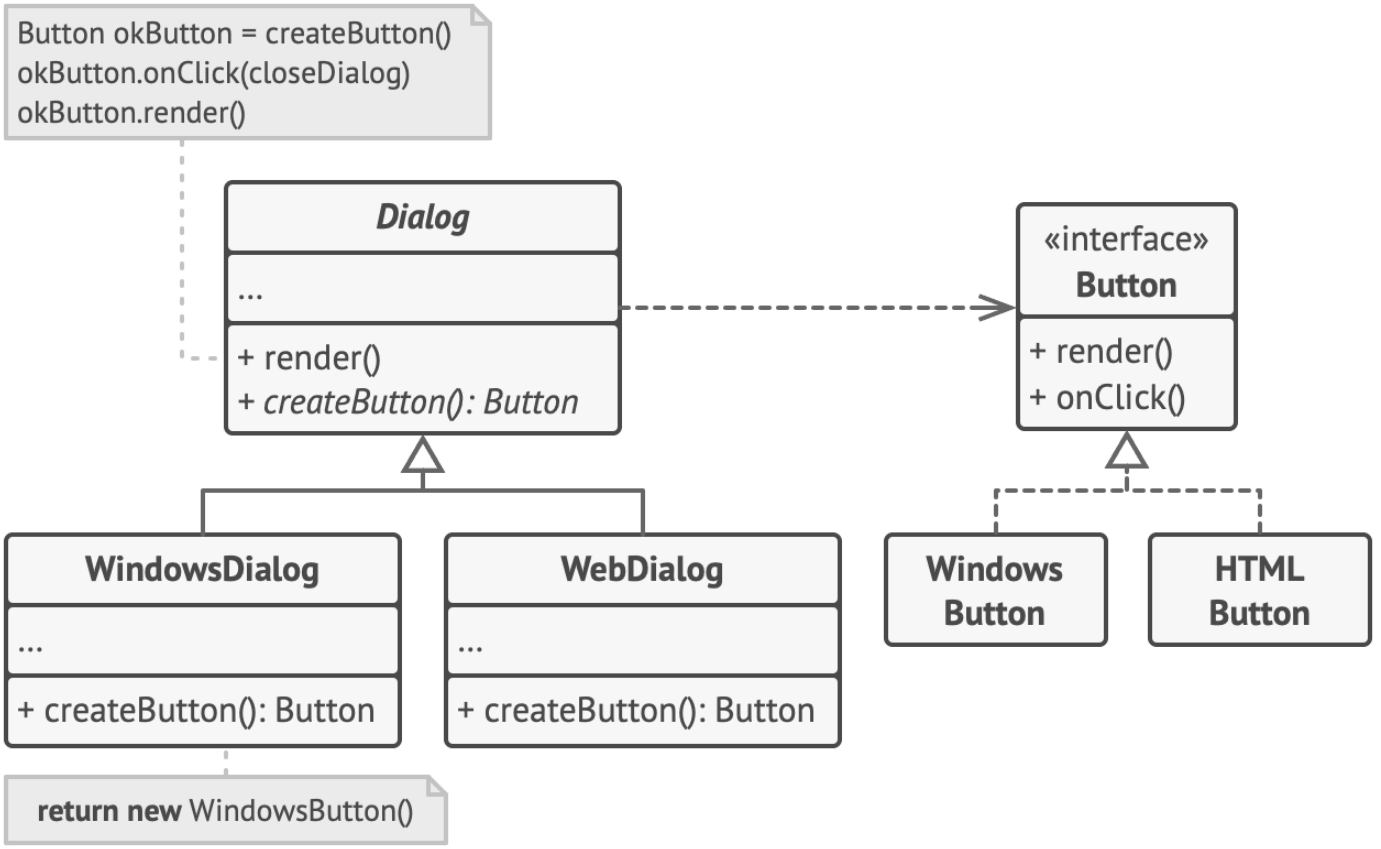
[4]**Конкретные создатели** по-своему реализуют фабричный метод, производя те или иные конкретные продукты.

Фабричный метод не обязан всё время создавать новые объекты. Его можно переписать так, чтобы возвращать существующие объекты из какого-то хранилища или кэша.

[2]**Конкретные продукты** содержат код различных продуктов. Продукты будут отличаться реализацией, но интерфейс у них будет общий.

Пример

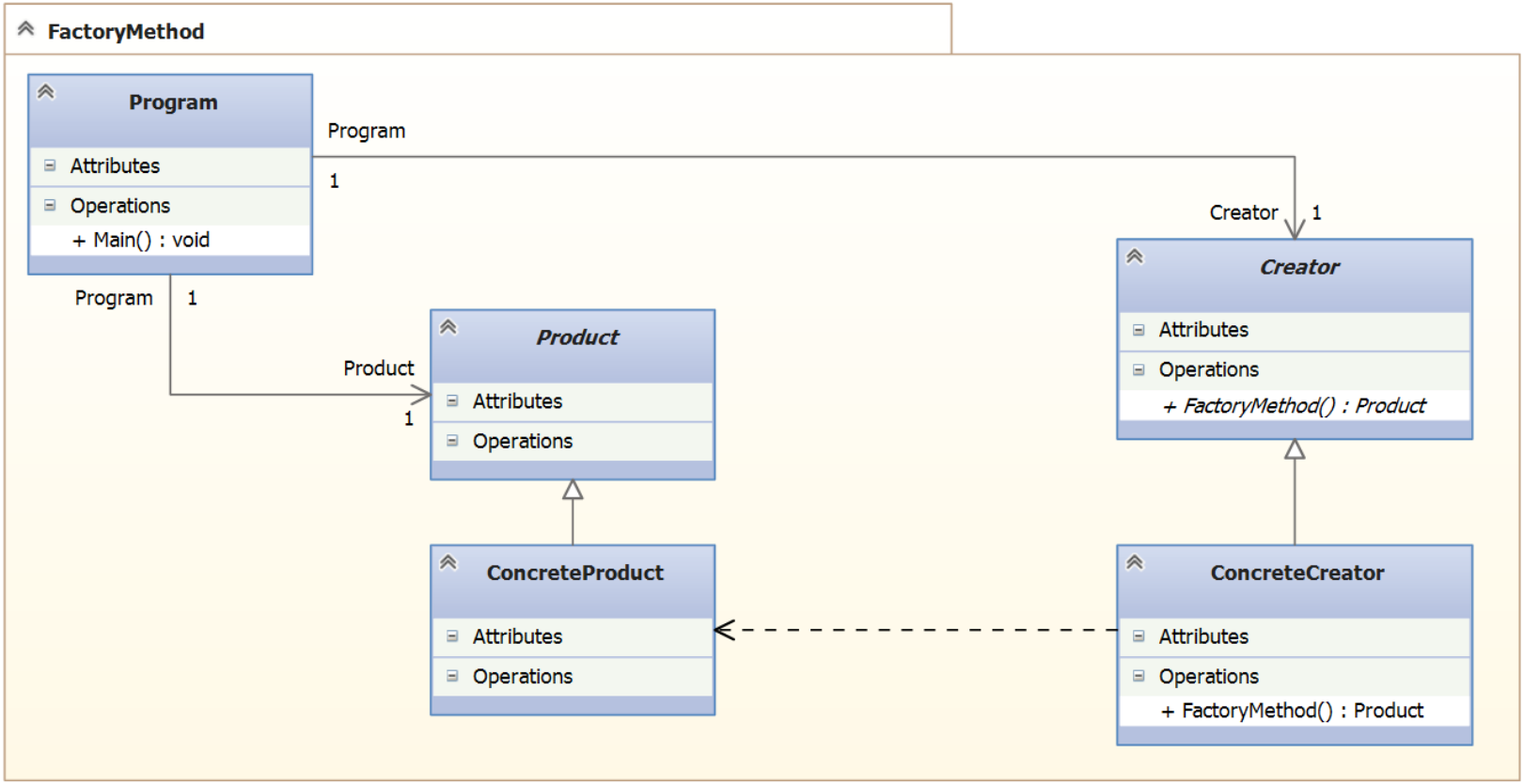
Фабричный метод может применяться для создания кросс-платформенных элементов интерфейса, не привязывая основной код программы к конкретным классам элементов.



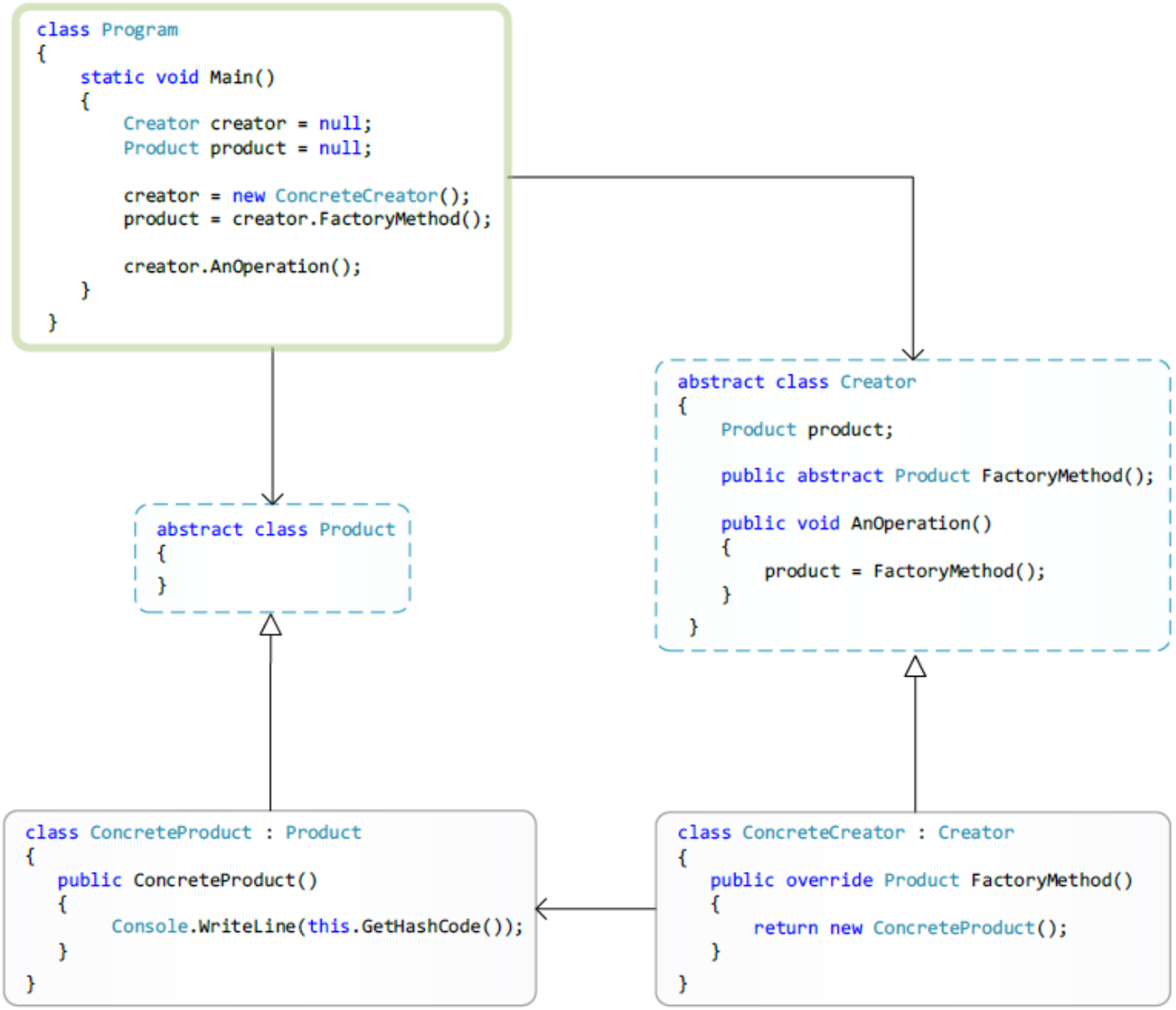
Фабричный метод объявлен в классе диалогов. Подклассы могут наследовать код из базового диалога, изменяя типы кнопок и других элементов, из которых базовый код строит окна графического пользовательского интерфейса.

Базовый класс диалогов работает с кнопками через их общий программный интерфейс. Базовый класс не зависит от конкретных классов кнопок, оставляя подклассам решение о том, какой тип кнопок создавать.

Структура паттерна на языке UML



Структура паттерна на языке C#



Варианты применения

- Абстрактному базовому классу Creator заранее неизвестно, экземпляры каких конкретных классов-продуктов потребуется создать.
- Абстрактный класс Creator спроектирован таким образом, чтобы объекты-продукты, которые потребуется создать, описывались производными от него классами (ConcreteCreator).
- Процесс создания продукта должен обеспечивать возможность получения различных вариаций создаваемого продукта.
- Абстрактный класс Creator планирует передать ответственность за создание объектов-продуктов одному из своих подклассов.
- Требуется собрать в одном месте (в группе наследников) всех ConcreteCreators ответственных за создание объектов-продуктов определенного типа.

Участники

Product - **Продукт**:
Предоставляет интерфейс для взаимодействия с продуктами.

Creator - **Создатель**:
Предоставляет интерфейс (абстрактные фабричные методы) для порождения продуктов. В частных случаях класс Creator может предоставлять реализацию фабричных методов, которые возвращают экземпляры продуктов (ConcreteProduct).

ConcreteProduct - **Конкретный продукт**:
Реализует интерфейс предоставляемый базовым классом Product.

ConcreteCreator - **Конкретная фабрика**:
Реализует интерфейс (фабричные методы) предоставляемый базовым классом Creator.

Отношения между участниками

Отношения между классами

- Класс ConcreteProduct связан связью отношения наследования с абстрактным классом Product.
- Класс ConcreteCreator связан связью отношения наследования с абстрактным классом Creator и связью отношения зависимости с классом порождаемого продукта ConcreteProduct.

Отношения между объектами

- Класс Creator предоставляет своим производным классам ConcreteCreator возможность самостоятельно выбрать вид создаваемого продукта, посредством реализации метода FactoryMethod.

Шаги реализации

1. Привести все создаваемые продукты к общему интерфейсу.

2. В классе, который производит продукты, создать пустой *фабричный метод*. В качестве возвращаемого типа указать общий интерфейс продукта.

3. Проинспектировать код класса и найти все участки, создающие продукты. Поочерёдно заменить эти участки вызовами фабричного метода, перенося в него код создания различных продуктов.

В фабричный метод, возможно, придётся добавить несколько параметров, контролирующих, какой из продуктов нужно создать.

4. Для каждого типа продуктов завести подкласс и переопределить в нём фабричный метод. Переместить туда код создания соответствующего продукта из суперкласса.

5. Если создаваемых продуктов слишком много для существующих подклассов создателя, можно ввести параметры в фабричный метод, которые позволят возвращать различные продукты в пределах одного подкласса.

6. **Если после всех перемещений** *фабричный метод* стал пустым, можно сделать его абстрактным. Если в нём что-то осталось —это будет его реализацией по умолчанию.

Преимущества и недостатки

+ Избавляет класс от привязки к конкретным классам продуктов.

+ Выделяет код производства продуктов в одно место, упрощая поддержку кода.

+ Упрощает добавление новых продуктов в программу.

+ Реализует принцип *открытости/закрытости*.

- Может привести к созданию больших **параллельных иерархий классов**, так как для каждого класса продукта надо реализовать свой подкласс создателя..

